



Loyola Marymount
University

CMSI 5350/EECE 5150 Machine Learning

Professor Lanyu Shang

Announcements

- PostCommit quiz today
 - Make sure your code was placed under `source/titanic.py`
- Homework 2 will be posted after today's class.
 - Due next Friday, 10/2

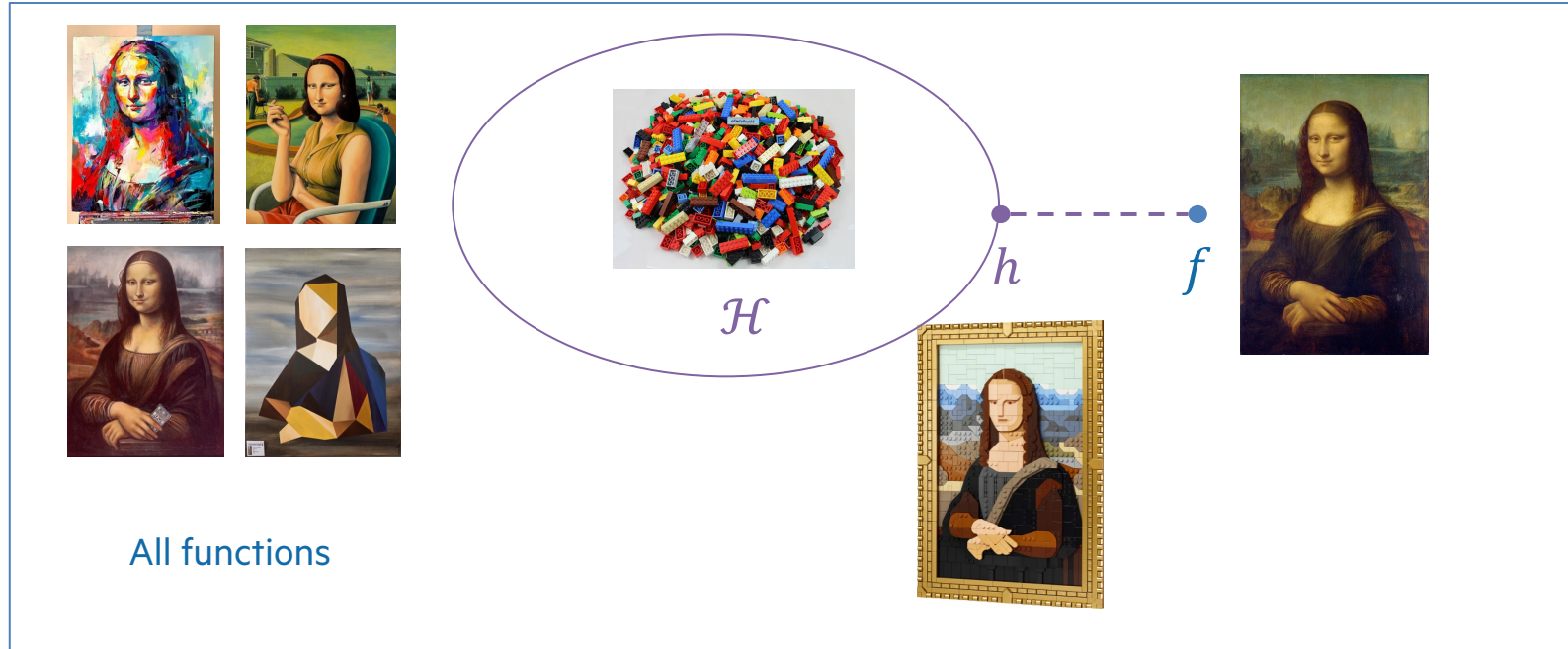
This lecture

- Decision tree
- Decision tree implementation with sklearn

Last lecture: ML as function approximation

- Problem setting
 - Set of possible **instances/samples** X
 - Set of possible **labels** Y
 - Unknown **target/ideal function** $f: X \rightarrow Y$
 - Set of **function hypotheses** $\mathcal{H} = \{h \mid h: X \rightarrow Y\}$
- Input: training data drawn i.i.d. from f
$$\mathcal{D}_n = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\}$$
- Output: hypothesis $h \in \mathcal{H}$ that best approximates f

ML as function approximation



Decision tree – problem setting

- Set of possible instances $X = \{x^{(1)}, x^{(2)}, \dots, x^{(n)}\}$
 - Each instance $x^{(i)} \in X$ is a feature vector
 - E.g. $x^{(i)} = \langle \text{humidity=low, wind=weak, outlook=rain, temp=hot} \rangle$
- Set of possible labels $Y = \{y^{(1)}, y^{(2)}, \dots, y^{(n)}\}$
 - Y can be discrete, e.g., $y^{(i)} = \{'yes', 'no'\}$
 - Y can be continuous, e.g., $y^{(i)} \in \mathbb{R}$
- Unknown target function $f: X \rightarrow Y$
- Set of function hypotheses $\mathcal{H} = \{h \mid h: X \rightarrow Y\}$
 - Each hypothesis h is a decision tree
 - Tree sort $x^{(i)}$ to leaf, which assigns $y^{(i)} \in Y$

Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No

Learning a decision tree

- Finding the optimal tree is computationally intractable.
 - Search space is exponential: 2^m nodes for m binary features
- ML: heuristic search approach (ID3 algorithm)
 1. Start from the **root**
 2. Find the non-terminal node that results in the “**best**” **split**
 3. Create new **descendant of node**, sort/split the data
 4. **Repeat** 2-3 until a **stopping criterion** is reached

How to find the
“best” split?

Example

- Step 1: $H(Y)$

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

$$H(X) = - \sum_{i=1}^n P(X = i) \log_2 P(X = i)$$

Example

- Step 2: info gain between Y and each feature.
 - 2.1 Windy

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

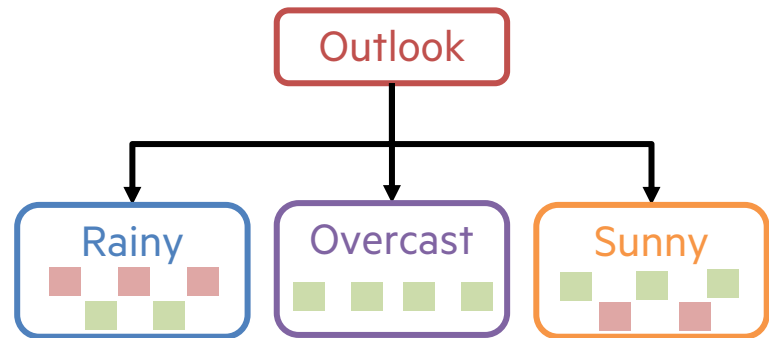
$$I(Y, A) = H(Y) - H(Y|A)$$

$$\begin{aligned} H(Y|A) &= \sum_{\alpha \in A} P(A = \alpha) H(Y|A = \alpha) \\ H(Y|A = \alpha) &= \\ &= - \sum_{y \in Y} P(Y = y|A = \alpha) \cdot \log_2 P(Y = y|A = \alpha) \end{aligned}$$

Example

- Step 2: info gain between Y and each feature.
 - $I(Y, \text{Outlook}) = \mathbf{0.246}$ ✓
 - $I(Y, \text{Temp}) = 0.029$
 - $I(Y, \text{Humidity}) = 0.151$
 - $I(Y, \text{Windy}) = 0.048$
- Outlook** as the root node
- Next, determine the next split for each node in ➡

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



Gini index

- Another approach to measure information gain.

$$1 - \sum_{i=1}^n P(X = i)^2$$

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini',  
splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1,  
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,  
max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None,  
ccp_alpha=0.0, monotonic_cst=None)
```

[\[source\]](#)

Parameters:

criterion : {"gini", "entropy", "log_loss"}, default="gini"

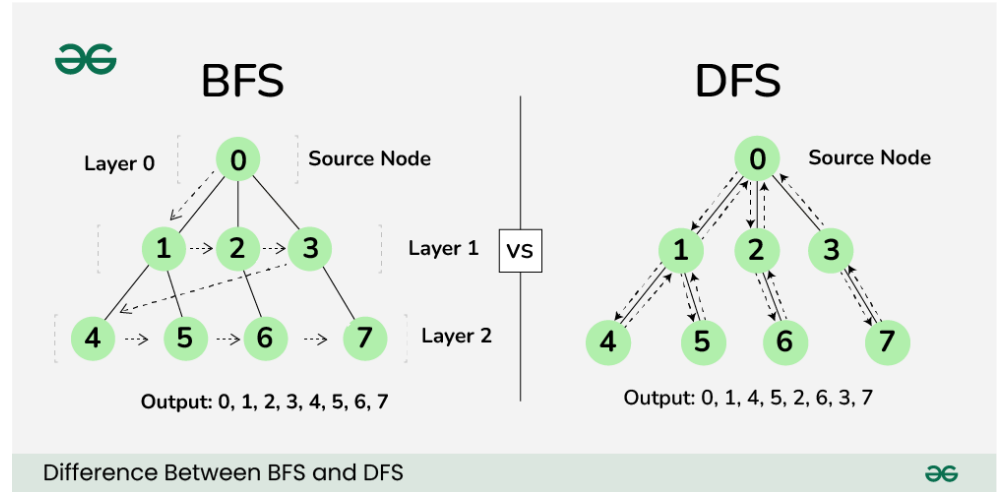
The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see [Mathematical formulation](#).

Decision tree

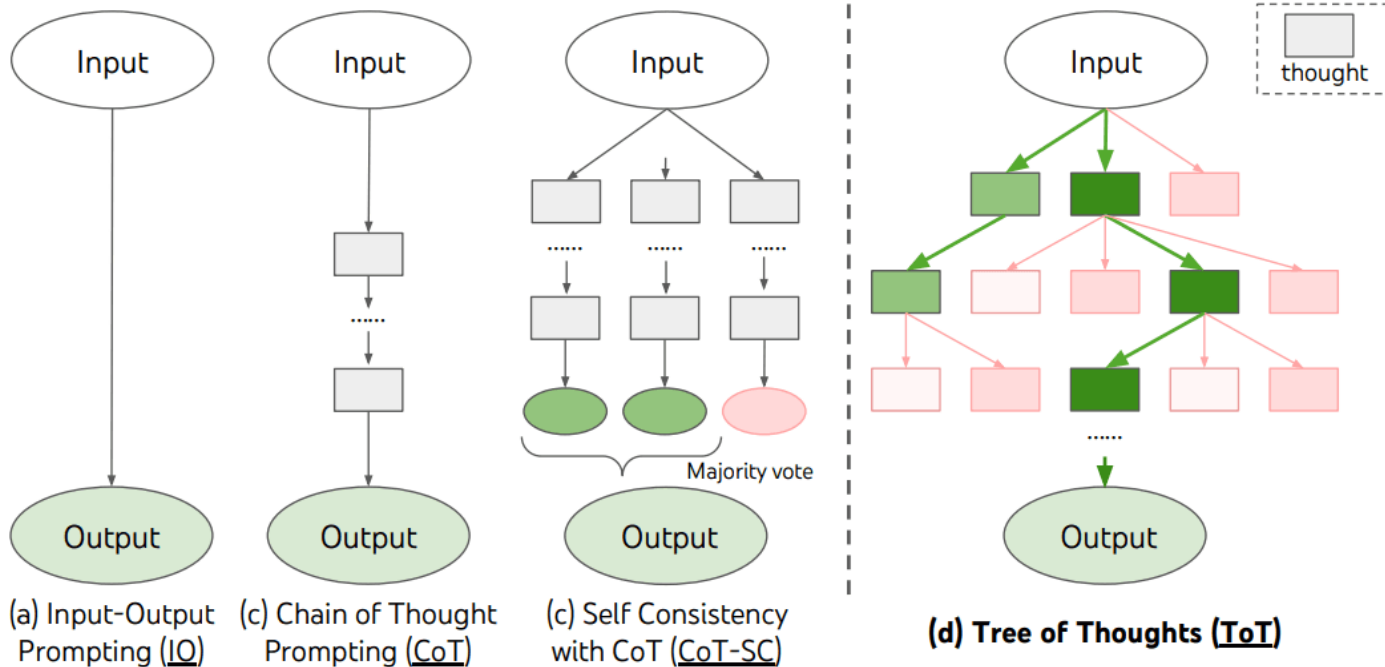
- Pros?
 - Interpretability - one of most intuitive classifiers
 - Works well with
 - Classification and regression
 - Non-linear relationships
 - Numerical and categorical **features**
- Cons?
 - Prone to overfitting
 - Instability - small changes in the training data can result in significantly different tree
 - Pruning / tuning can be tricky to get right

BFS or DFS?

- Training vs inference
- DFS, why?
 - Memory efficiency
 - Easy to implement the stopping criteria
 - Intuitive



ToT: tree of thoughts?



Colab notebook

DECISION TREE